

rsa

Neilan Ghosh

September 2026

DISCLAIMER: This write-up was written with the intent of explaining or walking one through the working of the code and my thoughts behind it. The paper is not meant to be mathematically rigorous.

1 Some basic definitions in the code

1.1 extGCD

First, we define the base case, where the code returns the GCD itself with the Bezout coefficients.

Then, we use recursive calls to get to the base case from any case using the Extended Euclidean algorithm, which is illustrated below. Although I will not be getting into the proof right now to keep the explanation concise.

$$\forall a, b, q, r \in \mathbb{Z}$$

$$\text{If } a = bq + r$$

$$\text{and } b = rq_1 + r_1$$

Then we can say that $\gcd(a, b) = \gcd(b, r)$.

Once we reduce to the base case (which is to make it simpler to calculate by passing $(b, a \bmod(b))$), the call stack unwinds by converting the coefficients (x_1, y_1) to (x, y) by the following step

$$x = y_1 \tag{1}$$

$$y = x_1 - \lfloor \frac{a}{b} \rfloor y_1 \tag{2}$$

This gives the true Bezout coefficients after unwinding the function calls.

1.2 invMod

For modular inverses, we use the extGCD function to find the gcd and Bezout coefficients. We pass the second Bezout coefficient to an unused variable. The logic on which this function works is as follows

for a modular inverse to exist, the gcd must be 1

$$\begin{aligned}ax + my &= 1 \\ax &= 1 - my \\ax &= (-y)m + 1 \\ax &\equiv 1 \pmod{m}\end{aligned}$$

Then the function returns $x \pmod{m}$ because x is essentially the inverse which we are looking for, but the function takes the remainder with m just in case $x < 0$ which gives a positive input.

1.3 powMod

Before understanding the current implementation of the function, we first need to know how decomposition works in exponents. For example,

$$3^{13} = 3^{(1011)_2} = 3^{(1*2^3 + 0*2^2 + 1*2^1 + 1*2^0)}$$

Essentially, we store the exponent as e , and then check its bits by using the bit-wise $\&$ operator. Whenever we encounter a 1, we multiply res with $a \pmod{m}$, and when we encounter a 0, we just take $a^2 \pmod{m}$. This technique is known as **modular exponentiation with binary exponentiation**, or "square and multiply" algorithm in short.

2 Primality Tests

2.1 Fermat's Little Theorem, Carmichael numbers

$$\begin{aligned}a^{p-1} &\equiv 1 \pmod{p} \\ \text{where } p &\text{ is such that } \gcd(a, p) = 1\end{aligned}$$

Fermat's Little theorem is obeyed by primes, and this can be used as a primality test. However, some composite numbers also pass the test and are thus termed as **Fermat pseudoprimes**. This is generally considered a weaker proof of primes.

There also exist numbers which give Fermat pseudoprimes with all bases coprime to n , and are known as the **Carmichael numbers**. Thus, there is a need for a stronger primality test to be employed in this RSA implementation, such as the **Miller-Rabin Primality Test**.

2.2 Miller-Rabin Primality Test, millerTest, primeCheck

The overall structure of the MR (Miller-Rabin) test to check the primality of an odd number n , is to first decompose $n - 1$

$$n - 1 = 2^s d \text{ where } d, s \in \mathbb{Z} \text{ and } d \text{ is odd}$$

Then, we take the d obtained from this decomposition and then conduct two checks:

$$\begin{aligned}
a^d &\equiv 1 \pmod{n} \\
&\text{and} \\
a^{2^r d} &\equiv -1 \pmod{n}
\end{aligned}$$

If the d obtained from the decomposition of n fails any of the tests, we have proof of n being composite for that base, and thus the function returns false (indicating that n is probably not a prime). Even if an n passes all rounds of testing, it is still treated as a probable prime.

Also, we run this test k times (given by the `effVal` argument taken in `primeCheck`), to give a degree of accuracy for the function. By repeating the test for k iterations, and by taking different bases every time, we are increasing the accuracy of the primality test, and also filtering out any Carmichael numbers if they are passed into the function.

Generally, $k = 20$, or $k = 40$ is used to make the primality test cryptographically secure in this implementation of RSA.

2.3 Prime Generation

For this implementation of RSA, I have kept 8 digit primes as a target for generation, since that should be secure enough for our purposes in this implementation, and are simpler to inspect as compared to truly cryptographically secure numbers.

8 digit numbers mean that we need 24 bits in binary, in which we must ensure that the most significant digit is turned on (or set to 1), and that the least significant digit is turned on as well (to ensure the generated number is odd).

3 Brief on RSA

How RSA works is that we have two sufficiently large primes, p and q . Then, we say that

$$\begin{aligned}
n &= p \cdot q \\
&\text{and} \\
\phi(n) &= (p - 1) \cdot (q - 1)
\end{aligned}$$

Now, we take an e , which is a number such that $\gcd(e, \phi(n)) = 1$ (generally 3 or 65537 or such numbers), and also d , which is given by the relation below

$$\begin{aligned}
de &\equiv 1 \pmod{\phi(n)} \\
d &\equiv e^{-1} \pmod{\phi(n)}
\end{aligned}$$

Once we get all the numbers we need, we can get to the actual encryption and decryption methods. Very simply put, the modular functions for encryption and decryption are as follows

$$\begin{aligned}
c &\equiv m^e \pmod{n} \\
&\text{and} \\
m &\equiv c^d \pmod{n} \\
\text{Given that } 0 &\leq m < n
\end{aligned}$$

4 The motivation behind RSA's choices

Throughout cryptography, we observe that in order for an encryption algorithm to be useful, it must pass a few criteria

1. The forward function i.e. encryption must be very quick and simple to calculate.
2. The backward function i.e. decryption must be difficult to calculate.

This can be achieved through what are called "trapdoors" - pieces of information which make the decryption of ciphertext very simple **if known**. Otherwise, decryption remains difficult.

We shall go through the elements one-by-one.

4.1 Public Keys and Private keys

RSA is an asymmetric encryption method, which means it has two keys. The public key is the tuple (n, e) and the private key is the tuple (n, d) . This implies that encryption can be done by anyone just by the public key.

The main logic behind RSA is best shown as follows

We need to find a d such that for a given number e

$$\begin{aligned}
e \cdot d &\equiv 1 \pmod{\phi(n)} \\
\exists k \in \mathbb{Z} &\text{ such that} \\
e \cdot d &= k \cdot \phi(n) + 1
\end{aligned}$$

Also, by Euler's Totient Rule, we can say that $m^{\phi(n)} \equiv 1 \pmod{n}$ given that $\gcd(m, n) = 1$

Therefore, in essence, we can say that $m^{e \cdot d} = m^{k \cdot \phi(n) + 1} = (m^{\phi(n)})^k \cdot m \equiv m \pmod{n}$

Essentially, on raising m to the power of $e \cdot d$, modulo n gives m itself, unchanged. This is a highly useful property, because it means that we can manipulate the process (mainly in the manufacturing of the number d), but retain the contents of the plaintext.

4.2 The trapdoor in RSA

This is where the numbers p and q get very important; they are sufficiently large primes, which are multiplied to get n . Also, we get $\phi(n)$ by $(p-1) \cdot (q-1)$. Now,

d is calculated as the inverse modulo of e with respect to $\phi(n)$. This means that when we know p , q , it is very simple to calculate $\phi(n)$, which then implies that it is faster to calculate d . Without the knowledge of p and q , and only knowing n , it would be very difficult to calculate d or even $\phi(n)$. Therefore, p and q behaves as a trapdoor in the case of RSA